

MindSync AI — Master System Documentation & Viva Handbook

End-to-End Architectural Breakdown, Codefile Index, ML/XAI Formulations, & Feature Guides

System Name: MindSync AI Wellness Companion	Target Audience: IT Professionals & Software Engineers
Frontend Framework: Flutter 3.x (Dart + Riverpod + Hive)	Backend Framework: FastAPI (Python 3.14 + Pydantic)
ML & XAI Engine: Random Forest + SHAP (Scikit-Learn)	GenAI Model: Llama 3.3 70B (via Groq LPUs)
Cloud Backend Host: Render Platform	Live Backend URL: https://ai-mental-wellness-companion-for-it.onrender.com/

1. System Technical Overview & System Architecture

MindSync AI is a full-stack digital mental wellness platform designed specifically for software developers and IT professionals. It bridges the gap between hardware physical tracking (pedometer step count), subjective mental health reporting (mood, stress, sleep), machine learning risk forecasting (Random Forest), explainable AI (SHAP), and conversational AI coaching (Llama 3.3 70B via Groq).

End-to-End System Architecture Pipeline:

```
+-----+ | 1. MOBILE
FRONTEND LAYER (FLUTTER) | | - UI Pages: Dashboard, Mood Logging, Burnout Predictor, AI Chatbot,
Reports | | - State Management: Riverpod (authStateProvider, dashboardProvider) | | - Hardware Access:
Pedometer (Step Counter) & Geolocator (Weather) | | - Local Storage: Hive Key-Value Storage (authBox,
settingsBox, moodSyncBox) |
+-----+-----+ | HTTP / REST
API (Bearer Firebase Token) v
+-----+ | 2. BACKEND
MICROSERVICES LAYER (FASTAPI) | | - Endpoints: /health, /api/predict/burnout, /api/recommendations/*,
/api/analytics | | - Security & Auth: JWT Bearer Validation + Fallback Payload Decoder | | - Firebase
Service: Firebase Admin SDK + Firestore Database Sync |
+-----+-----+ |
+-----+ v v +-----+
+-----+ | 3. ML & XAI SUBSYSTEM | | 4. GENERATIVE AI & ORCHESTRATION
| | - Model: Random Forest Classifier | | - Model: Llama 3.3 70B Versatile | | - Features: 9 Telemetry
Metrics | | - Infrastructure: Groq LPU Hardware | | - Explainability: SHAP Values | | - Fallback: IT
Wellness Rule System | +-----+
+-----+
```

2. Complete Codebase File Inventory & Technical Directory

The project is structured into two main subsystems: the **Backend (FastAPI)** and the **Frontend (Flutter)**.

File / Component Path	Layer / Purpose	Key Responsibilities & Functions
-----------------------	-----------------	----------------------------------

backend/app/main.py	Backend Entry	FastAPI app initialization, CORS middleware setup, router mounting for all API endpoints.
backend/app/core/security.py	Security	JWT Bearer token verification via Firebase Admin SDK + JWT payload fallback decoding for Render.
backend/app/core/firebase.py	Firebase Core	Firebase Admin SDK initialization with explicit projectId 'mindsync-ai-18fbb' and Firestore client.
backend/app/ml/pipeline.py	ML Pipeline	BurnoutPipeline class. Loads pre-trained Random Forest model, runs inference, and computes SHAP feature importance.
backend/app/ml/recommendation_engine.py	ML Recommendations	HybridRecommendationEngine class. Combines rule-based health checks + ML weights + Groq AI advice.
backend/app/services/ai_orchestrator.py	AI Orchestration	AIOrchestrator class. Executes Groq Llama 3.3 70B API calls with retry backoff and local caching.
backend/app/api/endpoints/predict.py	API Endpoint	POST /api/predict/burnout. Accepts 9-feature telemetry payload and returns ML burnout prediction + SHAP factors.
backend/app/api/endpoints/recommendations.py	API Endpoint	POST /api/recommendations/generate & /daily-summary. Generates personalized developer habits and summaries.
backend/app/api/endpoints/analytics.py	API Endpoint	POST /api/analytics/report/export. Generates formatted PDF binary stream report for users.
frontend/lib/main.dart	Frontend Entry	Flutter app entry point, Firebase init, Riverpod ProviderScope wrap, global error catch guards.
frontend/lib/core/config/app_config.dart	App Config	Reads BACKEND_URL from .env (defaults to Render URL) and manages feature flags.
frontend/lib/core/network/dio_client.dart	Network Layer	Configures Dio HTTP client with base URL, timeouts (15s), and mounts AuthInterceptor.
frontend/lib/core/network/auth_interceptor.dart	Network Security	Intercepts outgoing HTTP calls, retrieves Firebase ID token from Hive authBox, and attaches Bearer header.
frontend/lib/core/services/pedometer_service.dart	Hardware Service	Subscribes to hardware Pedometer stepCountStream with smooth background step simulation fallback.
frontend/lib/features/authentication/	Auth Feature	Firebase authentication (SignIn, Register, Password Reset, Email Verification) with token persistence.
frontend/lib/features/chat/	Chat Feature	Empathetic AI Chatbot interface. Calls Groq Llama 3.3 with crisis safety overrides & IT wellness fallbacks.
frontend/lib/features/reports/	Reports Feature	PDF & CSV report export history panel. Uses OpenFilex & path_provider to auto-open files upon download.

3. Comprehensive User Guide for All System Features

- **Dashboard & Hardware Pedometer Tracking:** When the app opens, the Dashboard displays real-time biometric metrics. The hardware Pedometer automatically counts steps in the background using device accelerometer sensors and updates the daily step progress bar (Goal: 10,000 steps). Weather data is fetched using local geolocation coordinates.

- **Daily Mood & Biometric Check-In:** Users tap 'Log Mood' to record their daily psychological state (1-10 scale: Stressed, Stable, Happy, Anxious, Exhausted), sleep duration, hydration (ml/glasses), and active exercise minutes. Data is synced to Firebase Firestore and cached locally in Hive.
- **ML Burnout Risk Predictor:** Users navigate to the Burnout Predictor screen and submit their daily metrics. The app sends the 9-feature telemetry vector to the backend Random Forest classifier, returning an instant Burnout Risk % (Low, Medium, High) alongside SHAP Explainable AI contributing factors (e.g. '+35% risk from 13h work day').
- **Empathetic AI Chatbot:** Users can start an interactive conversation with MindSync AI. The chatbot utilizes Llama 3.3 70B to provide developer-specific wellness advice (managing pull request stress, screen breaks, sleep hygiene). If crisis keywords like 'suicide' are detected, the app immediately overrides with 988 Crisis Lifeline resources.
- **PDF & CSV Report Export:** Users can generate comprehensive weekly or monthly PDF/CSV health summaries. Upon download, the app uses OpenFilex and path_provider to automatically open the generated PDF document directly on the user's mobile device.

4. Mathematical Formulations & ML Pipeline

Random Forest Classifier Equations:

1. Entropy: $H(S) = - \sum [p_i * \log_2(p_i)]$
2. Information Gain: $IG(S, A) = H(S) - \sum [(|S_v| / |S|) * H(S_v)]$
3. Gini Impurity: $Gini(S) = 1 - \sum (p_i)^2$

SHAP Explainable AI (Shapley Value Formulation):

$$\phi_i(v) = \sum [(|S|! * (|N| - |S| - 1)!) / |N|!] * [v(S \cup \{i\}) - v(S)]$$

5. Viva Presentation Cheat Sheet & Examiner Q&A; Guide

30-Second Elevator Pitch:

"MindSync AI is an intelligent mental wellness shield designed for software developers. It collects biometric telemetry like sleep, steps, and stress, passes them through a Random Forest Machine Learning classifier to predict burnout risk, uses SHAP Explainable AI to pinpoint exact root causes, and leverages Groq-accelerated Llama 3.3 LLMs to deliver real-time actionable developer coaching."

Q1: How does authentication work between Flutter and FastAPI on Render?

Answer: Flutter uses Firebase Auth to log in users. Upon login, the Firebase ID token is persisted in Hive. Every outgoing Dio HTTP request attaches this token as a Bearer header. FastAPI verifies the token via Firebase Admin SDK, with a JWT payload fallback decoder if service account keys are absent on Render.

Q2: Why did you use Random Forest instead of a Deep Learning Neural Network?

Answer: Random Forest provides superior classification accuracy on tabular biometric data of moderate sample size without overfitting. More importantly, tree ensembles integrate directly with SHAP for exact Explainable AI attribution, whereas neural networks act as opaque black boxes.

Q3: How does the step counter work automatically?

Answer: The app uses pedometer_service.dart which listens to device accelerometer sensors via Pedometer.stepCountStream. If hardware sensors are unavailable (such as in emulators), it smoothly falls back to a realistic step simulation engine so the UI never crashes.

Q4: How do you handle API key failures or offline mode?

Answer: The app has a multi-tier fallback architecture. If Groq API keys are unconfigured or network requests fail, the app identifies placeholder patterns ('your_', 'mock_') and seamlessly falls back to local intelligent IT wellness rules and Hive cached data.

Q5: How does report auto-opening work after export?

Answer: Upon downloading a PDF or CSV report, the app saves the file to local device storage using path_provider and immediately invokes OpenFilex.open(file.path) to trigger the device's native PDF reader automatically.